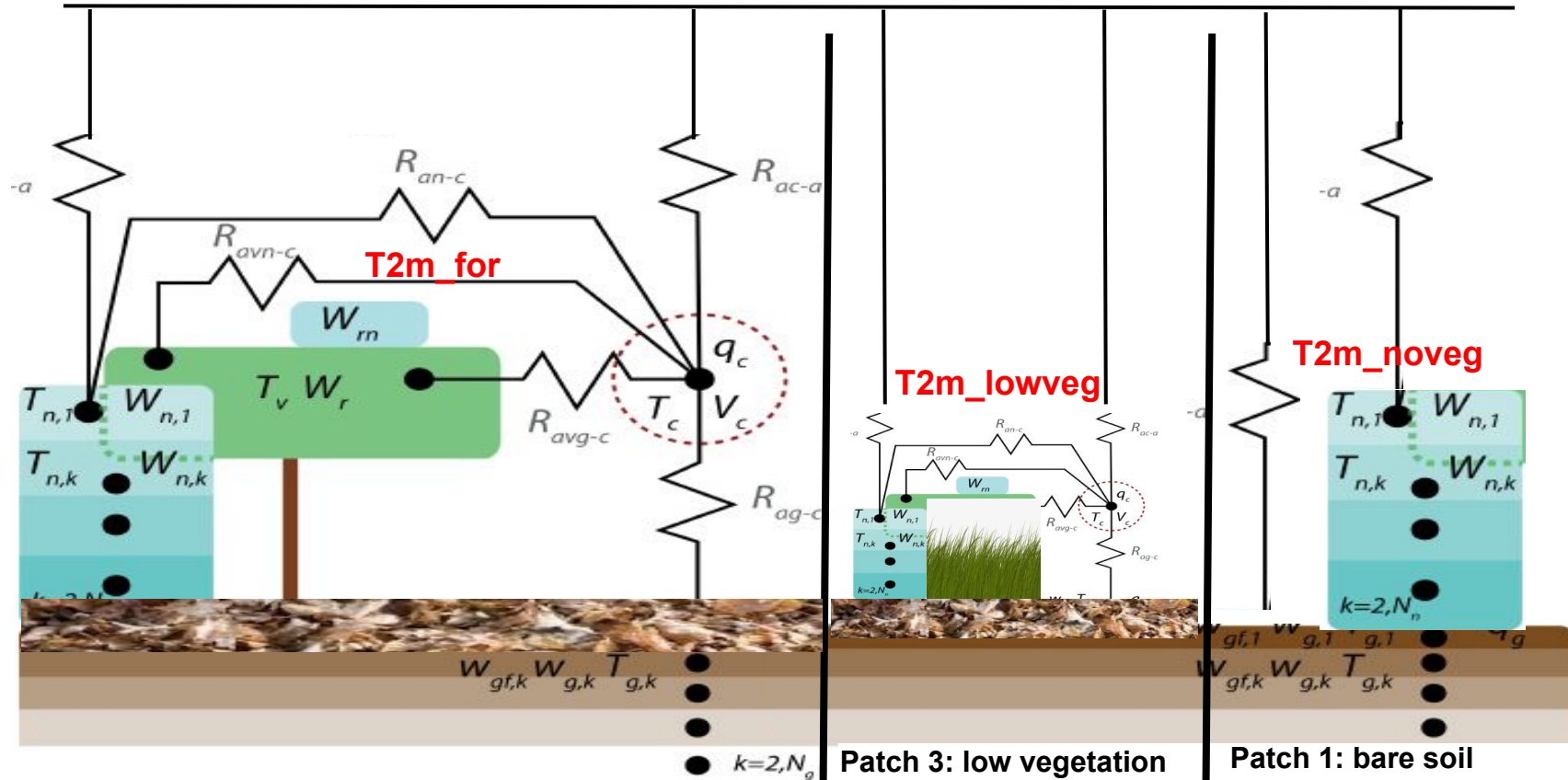


OSVAS: Offline SURFEX Validation System

Samuel Viana, AEMET

Autumn Surface Working Week 2025, November 24-28, online

New “target” surface configuration with 3 patches:



Patch 2: forest (diffusion soil, explicit snow, MEB, litter + newMLCH (Increased SOC% for 1st and 2nd layer))

Patch 3: low vegetation (diffusion soil, explicit snow, MEB, newMLCH DSL)

Patch 1: bare soil (diffusion soil, explicit snow, DSL)

ACCORD WW on multi-layer surface physics at RMI in Brussels, December 2024



We decided to create a shared tool for validation of new surfex setups currently under testing in the ACCORD community.

Adrien Napoly (GMAP, Météo-France)
Aaron Boone (SURFEX team, Météo-France)
Jan de Pue (RMI)
Samuel Viana (AEMET)
Patrick Samuelsson (SMHI)

OSVAS: Offline-SURFEX Validation System

[OSVAS](#) is a tool recently developed within the ACCORD community to automate the full process of validating SURFEX configurations through OFFLINE, single point SURFEX simulations.

- It aims at providing a systematic approach for testing NWP-like SURFEX settings in an OFFLINE SURFEX environment over specialized Atmospheric & Ecosystem stations from the ICOS project
- It also facilitates a systematic & homogeneous validation of the tested configurations across different sites using flux data and/or soil state/screen level variables from the same source.
- It covers the entire cycle: Forcing → Simulation → Extraction → Validation/Verification → Visualization

develop

2 Branches 0 Tags

Go to file



Add file

<> Code



svianaj -Extra documentation in yaml files

1bc5282 · 5 days ago

87 Commits

config_files	-Extra documentation in yaml files	5 days ago
docs	Update step4_nc2sqlite.md	last week
harp_output	Different changes to the scripts to generate FCTABLES & ...	6 months ago
namelists	- Fix installation of correct yq version on ATOS	last week
renv_atos	- Improve installation on ATOS, including isolating a HAR...	5 days ago
scripts	-Extra documentation in yaml files	5 days ago
sqlites	- Align units in obstables to units in netcdf output files	3 weeks ago
.gitignore	Add tentative separate namelists for the sites	8 months ago
README.md	Update README.md	last week
icos_cookie.txt	- Fix main script to run validation for dates different to r...	last week
requirements.txt	- Fix installation of correct yq version on ATOS	last week

README



OSVAS — Offline Surfex Validation System

OSVAS is a workflow developed within the **ACCORD community** to automate the full SURFEX offline validation chain:

- **Generate** SURFEX forcing data from ICOS atmospheric datasets
- **Download & process** ICOS flux data for validation
- **Run** SURFEX OFFLINE simulations (PGD, PREP, OFFLINE)

About

Offline Surfex Validation System

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Packages

No packages published

Languages

Jupyter Notebook 99.4% Other 0.6%

Strategy for validation

- Model output & ICOS observations must be interpolated and / or converted to a suitable format for data analysis.
- HARP software solved this with FCTABLES and OBSTABLES in sqlite format.

fcst_dttm	lead_time	SID	z	lat	lon	valid_dttm	parameter	units	AIBC_ibera_det
1466467200	0	4300000003	0	44.4936	-0.95608	1466467200	H	surface	-30.37969687145902
1466467200	1	4300000003	0	44.4936	-0.95608	1466470800	H	surface	-5.688453167427961
1466467200	2	4300000003	0	44.4936	-0.95608	1466474400	H	surface	-5.6088438947458785
1466467200	3	4300000003	0	44.4936	-0.95608	1466478000	H	surface	-6.269733254434777
1466467200	4	4300000003	0	44.4936	-0.95608	1466481600	H	surface	-5.465673226949579

- OSVAS produces these type of files & HARP's verification scripts have been integrated in OSVAS as an initial validation tool for simple diagnosis.
- Other more specialized surface validation scripts developed within OSVAS or imported from other tools must be adapted to read observations & model output from these sqlite files

OSVAS installation

- First, get a copy (git clone) of OSVAS and oper-harp-verif scripts:

<https://github.com/ACCORD-NWP/OSVAS>

<https://github.com/harphub/oper-harp-verif>

- The installation is isolated within a `conda` environment, so make sure to have it installed in your system (or load the conda module on ATOS).
- All installation steps are managed by an installation script in **`$OSVAS/scripts/bash_scripts/create_conda_environment.sh`**
- On ATOS, the script also takes care of installing HARP in a Renv setup (takes ~45min!!).
- To install HARP manually in your R setup, follow [this](#) and [this](#)

OSVAS Steps:

1. Retrieve ICOS data & create the forcing files
2. Retrieve ICOS observations for validation
3. Run SURFEX OFFLINE (PGD, PREP, OFFLINE)
4. Extract variables to validate from the model output
5. Run verification / validation tools (currently HARP)
6. Visualize results (HARP's shiny apps)

OSVAS Steps:

1. Retrieve ICOS data & create the forcing files (jupyter notebook)
2. Retrieve ICOS observations for validation (jupyter notebook)
3. Run SURFEX OFFLINE (PGD, PREP, OFFLINE) (SURFEX binaries)
4. Extract variables to validate from the model output ([nc2sqlite.py](#))
5. Run verification / validation tools (currently HARP) (oper-harp-verif)
6. Visualize results (HARP's shiny apps) (shiny web apps)

OSVAS main script

- Currently all the different OSVAS steps are run from a bash script capable to run the jupyter notebooks, python tools, SURFEX and HARP R scripts:

`$OSVAS/scripts/bash_scripts/surfex_OSVAS_run_linux.sh`

`$OSVAS/scripts/bash_scripts/surfex_OSVAS_run_atos.sh`

- The workflow is controlled from a yml file (per station) read by this script:

`$OSVAS/config_files/Stations/${STATION_NAME}.yaml`

- The only editing needed is:
 - When installing OSVAS: path to your OSVAS, HARP & SURFEX setups, and (if needed) update the paths to the physiography data.
 - For every OSVAS run: **`$STATION_NAME`** to locate the yml file

```
##### OSVAS #####
##### (OFFLINE SURFEX VALIDATION SYSTEM) #####
##### STEP 0: Define Station, relevant paths and which steps to run from the yaml file #####
#####
# Select name of the Station where to run the simulation, and the experiment name to import the yaml
# configuration, namelist, etc Currently select between Majadas_del_tietar (ES), Meteopole(FR),
# Loobos(NL). Make sure you have a recent token for the ICOS API stored in $OSVAS/icos_cookie.txt
```

```
export STATION_NAME=Majadas_del_tietar
export OSVAS=/home/pn56/OSVASgh/ #SET PATH TO YOUR OSVAS SETUP
export HARP=/home/pn56/operharpverif/ #SET PATH TO HARP SCRIPTS
yaml_file="$OSVAS/config_files/Stations/${STATION_NAME}.yaml"
```

(...)

```
#####
#### STEP 3: CONFIGURE AND RUN THE SIMULATIONS #####
#### Set path to the SURFEX setup & physiographic files, read experiments/namelists to import #####
#### make run & output paths, make useful links (forcing & physiographic files, modify dates #####
#### in the namelists, run SURFEX for each experiment#####
#####
# Define path of SURFEX code and SURFEX executables, add to $PATH
```

```
SURFEX_PARENT=$HOME
SURFEX_VER=SURFEX_NWP
SURFEX_HOME=$SURFEX_PARENT/$SURFEX_VER #PATH TO THE SURFEX SETUP
SURFEX_PROFILE=profile_surfex-LXgfortran-SFX-V8-1-1-NOMPI-OMP-02-X0
SURFEXEXE=$SURFEX_HOME/src/dir_obj-LXgfortran-SFX-V8-1-1-NOMPI-OMP-02-X0/MASTER/ #PATH TO SURFEX BINS
```

```
# Add these to the $PATH
export PATH=${SURFEXEXE}:$PATH
```

Physiographic files

- ECOCLIMAP I / II , hardcoded soil texture, SOC, orography...

`$OSVAS/scripts/bash_scripts/surfex_OSVAS_run_linux.sh`

```
#SET PATH TO YOUR PHYSIOGRAPHY FILES
PARAMFILES=${SURFEX_HOME}/MY_RUN/ECOCLIMAP/ # ECOCLIMAP param/bin files
DIRFILES=$HOME/PHYSIO/ # hdr/dir files of ECOCLIMAP version in the namelists
```

- ECOCLIMAP-SG, SOILGRIDS, GMTED orography, SOC files, etc...

`$OSVAS/scripts/bash_scripts/surfex_OSVAS_run_atos.sh`

```
#SET PATH TO YOUR PHYSIOGRAPHY FILES
HM_CLDATA=/ec/res4/hpcperm/hlam/data/climate
E923_DATA_PATH=$HM_CLDATA/E923_DATA
PGD_DATA_PATH=$HM_CLDATA/PGD
ECOSG_DATA_PATH=$HM_CLDATA/ECOCLIMAP-SG
GMTED2010_DATA_PATH=$HM_CLDATA/GMTED2010
SOILGRID_DATA_PATH=$HM_CLDATA/SOILGRID
ECOSG_COVERS=$ECOSG_DATA_PATH/COVER
#The following 2 physiography sources must come from a harmonie setup:
GMTED_PATH=/ec/res4/scratch/sp3c/hm_home/harmonie46h111/climate/DKCOEXP/
SOILGRIDS_PATH=/ec/res4/scratch/sp3c/hm_home/harmonie46h111/climate/DKCOEXP/
```

The yml configuration file

\$OSVAS/config_files/Stations/\${STATION_NAME}.yml

OSVAS_steps:

Create_forcing: true

Get_validation: true

Run_surfex: True

Surfex_steps: #Don't forget a space before each item

- pgd
- PREP
- OFFLINE

Expnames: #Don't forget a space before each item

- MEBREFOL

Extract_model_sqlites: true

Run_HARP: true

Display_HARP: true

Station_metadata:

Station_name: Majadas_del_tietar

SID: 4300000005

elev: 265.0

lat: 39.94033

lon: -5.77465

vegtype: 19

- Activate/deactivate OSVAS & SURFEX steps, select experiments to run
- For each EXPNAME, a namelist is expected in

**\$OSVAS/namelists/\$STATION_NAME/
OPTIONS.nam_\$EXPNAME**

- Station metadata used by step 2 and 4 to fill columns in OBSTABLE*.sqlite and FCTABLES*.sqlite files

The yml configuration file

\$OSVAS/config_files/Stations/\${STATION_NAME}.yaml

```
Forcing_data:
  height_T: 2                # Height of the temperature measurement
  height_V: 10               # Height of the windspeed measurement
  run_start: '2021-5-01 00:00:00' # Timestamp for the forcing start
  run_end: '2022-12-31 23:30:00'  # Timestamp for the forcing end
  forcing_format: 'ascii'        # Choose between netcdf or ascii
dataset1:
  doi: https://meta.icos-cp.eu/objects/fPAqntOb1uiTQ2KI1NS1CH1B
  timedelta: 30 # in minutes
  variables:
    Forc_CO2: -, 0.00062
    Forc_PS: PA, *1000
    Forc_RAIN: P, /(timedelta*60)
    Forc_SNOW:
    Forc_WIND: WS
    Forc_DIR: WD
    Forc_DIR_SW: SW_IN
    Forc_LW: LW_IN
    Forc_QA:
    Forc_SCA_SW:
    Forc_TA: TA, +273.15
    Forc_RH: RH
```

**!! Currently SURFEX_NWP is not
producing a correct NC output with
NC forcing files**

- Define forcing format & dates, height of the forcing, and ICOS datasets from where to extract each forcing variables
- Each forcing variable is assigned a column from the ICOS dataset. Unit transformations can be specified.
- If several datasets with different sampling rates are provided, the data will be upsampled to a common (smallest) timedelta.
- ASCII: Forc_*.txt files + Params_config.txt
- NetCDF: [FORCING.nc](#) merged from daily nc files

Forcing files are stored in: **\$OSVAS/forcings/Stations/\$STATION_NAME/**

The yml configuration file

\$OSVAS/config_files/Stations/\${STATION_NAME}.yml

```
Validation_data:
  validation_start: '2022-1-01 00:00:00'
  validation_end: '2022-12-31 23:30:00'
  common_obstable: FALSE # Write obs to a common obstable or to a single-station one
dataset1:
  doi: https://meta.icos-cp.eu/objects/fPAqnt0b1uiTQ2KIiNS1CHlB
  timedelta: 30
  variables:
    SW_OUT: SW_OUT
    LW_OUT: LW_OUT
    SW_IN: SW_IN
    LW_IN: LW_IN
    TS_1: TS_1
    TS_2: TS_2
    SWC_1: SWC_1
    SWC_2: SWC_2
  units: # If compatible, these should be similar to units attribute in output
  #netcdf files. Otherwise a conversion in set_params.R will be needed
  SW_OUT: W/m2
  LW_OUT: W/m2
  LW_IN: W/m2
  SW_IN: W/m2
  TS_1: degC
  TS_2: degC
  SWC_1: percent
  SWC_2: percent
dataset2:
  doi: https://meta.icos-cp.eu/objects/tONKGY9p0YqVInayCYac-4LI
  timedelta: 30
  variables:
    H: H
    LE: LE
  units:
    H: W/m2
    LE: W/m2
```

- Define validation period & ICOS datasets with validation data
- Map ICOS dataset variables (right) to their desired name (left) in the **OBSTABLE_YYYY.sqlite** files
- Set variable units. If possible their format should be similar to the “units” attribute of the variable in output netcdf files.
- Observations can be written to common obstables for all stations, or to single-station yearly files:
\$OSVAS/sqlites/validation_data/common_obstables/
\$OSVAS/sqlites/validation_data/\$STATION_NAME/

Steps 1&2: Retrieve ICOS observations & create the forcing & validation files

- To download ICOS data using the python [icoscp](#) API, you need to create an account at the [ICOS carbon portal](#), and put the ICOS token in \$OSVAS/icos_cookie.txt. You'll find it at the bottom of your user profile:

API token	
Value	cpauthToken=WzE3NjM3Mjk0MDA3NjUsInN2aWFuYWpAYWVtZXQuZXMiLCJPcmNpZCJdHjBmAjEakC37ctR0MtWiKPu9eeRvhTBWfv9Q8ygiKh27yCSP5x5eu4W878guoKM2+AyLFR+1AjEA0o/w2VCymCWwTka39aWbK4/hiY5iyrBnXkjAZ8Sp6oOV1ek4YMTj+ir3lXCgSNp
Expiry	2025-11-21T12:50:00.765Z
Source	Orcid

- Forcing files are stored in: **\$OSVAS/forcings/Stations/\$STATION_NAME/**. If previous forcing files are present, you'll see a message asking to delete a .lockfile first (ASCII) or the existing daily .nc files (Netcdf)
- Validation data is written without checking for duplicates, so make sure not to retrieve data for the same period twice!

Step 3: Run SURFEX OFFLINE steps

- The main control script goes through each EXPNAME, prepares the namelist, runs the selected SURFEX steps at \$RUNDIR and moves output and logs to \$OUTDIR:
`RUNDIR=$OSVAS/RUNS/$STATION_NAME/$EXPNAME/run/`
`OUTDIR=$OSVAS/RUNS/$STATION_NAME/$EXPNAME/output/`
- On ATOS, it is recommended to submit the main script as a job to the HPC queue. The script's header contains the needed SLURM directives. Just make sure to update the paths for the output & error logfiles!

Step 4: Sqlite extraction of model data: nc2sqlite.py

```
usage: nc2sqlite.py [-h] -p PARAM_DICT -s STATION_LIST -st STATION -o OUTPUT -m EXPERIMENT_NAME ncdir
```

Convert point NetCDF SURFEX output to SQLite.
The script will create a EXPERIMENT_NAME folder in the OUTPUT path, which will be populated with monthly FCTABLES for every variable in the PARAM_LIST, stored in YYYY/MM subfolders

positional arguments:

ncdir Directory containing NetCDF files

options:

-h, --help show this help message and exit

-p PARAM_DICT, --param_list PARAM_DICT
 Path to param_dict.json

-s STATION_LIST, --station_list STATION_LIST
 Path to station_list_default.csv

-st STATION, --station STATION
 Station ID

-o OUTPUT PATH, --output OUTPUT PATH
 Output base directory

-m EXPERIMENT_NAME, --experiment_name EXPERIMENT_NAME
 Experiment name

\$OSVAS/scripts/nc2sqlite/param_dict.json :

```
{
  "_comment": "Parameter mapping between SURFEX
variable names (left column) in netcdf files and
observation variable names",
  "RI_ISBA": "RI",
  "H_ISBA": "H",
  "LE_ISBA": "LE",
  "WG1_ISBA": "SWC_1",
  "WG2_ISBA": "SWC_2",
  "WG3_ISBA": "SWC_3",
  "WG4_ISBA": "SWC_4",
  "WG5_ISBA": "SWC_5",
  "WG6_ISBA": "SWC_6",
  "WG7_ISBA": "SWC_7",
  "WG8_ISBA": "SWC_8",
  "TG1_ISBA": "TS_1",
  "TG2_ISBA": "TS_2",
  "TG3_ISBA": "TS_3",
  "TG4_ISBA": "TS_4",
  "TG5_ISBA": "TS_5",
  "TG6_ISBA": "TS_6",
  "TG7_ISBA": "TS_7",
  "TG8_ISBA": "TS_8",
  "LWU_ISBA": "LW_OUT",
  "LWD_ISBA": "LW_IN",
  "SWU_ISBA": "SW_OUT",
  "SWD_ISBA": "SW_IN"
}
```

\$OSVAS/sqlites/station_list_SURFEX.csv :

```
"SID","lat","lon","elev","name"
4300000001,64.2500,19.7666,267,"Svartberget"
4300000002,56.1000,12.4166,115,"Hyltemossa"
4300000003,44.4936,-0.95608,40,"Bilos"
4300000004,60.08333,17.48333,46,"Norunda"
4300000005,39.940556,-5.774722,258,"Majadas_del_tietar"
4300000006,43.572857,1.37474,158,"Meteopole"
4300000007,52.1664833,-5.7435194,25,"Loobos"
```

**!! Make sure this info is similar to
Station_metadata in the .yaml file!!**

Step 4: Sqlite extraction of model data: nc2sqlite.py

Example of a sqlite file (FCTABLE_LE_202202_00.sqlite) extracted for LE:

fcst_dttm	lead_time	z	SID	lat	lon	valid_dttm	parameter	units	MEBREFOL_det
1643673600	0	258	4300000005	39.940556	-5.774722	1643673600	LE	W/m2	0.5348008842360477
1643673600	0.5	258	4300000005	39.940556	-5.774722	1643675400	LE	W/m2	0.38031965143080404
1643673600	1	258	4300000005	39.940556	-5.774722	1643677200	LE	W/m2	0.6053065853882629
1643673600	1.5	258	4300000005	39.940556	-5.774722	1643679000	LE	W/m2	0.4973059382072318
1643673600	2	258	4300000005	39.940556	-5.774722	1643680800	LE	W/m2	0.523229212078078
1643673600	2.5	258	4300000005	39.940556	-5.774722	1643682600	LE	W/m2	0.471013431759338
1643673600	3	258	4300000005	39.940556	-5.774722	1643684400	LE	W/m2	0.3331034463327768
1643673600	3.5	258	4300000005	39.940556	-5.774722	1643686200	LE	W/m2	0.3618945744552383
1643673600	4	258	4300000005	39.940556	-5.774722	1643688000	LE	W/m2	0.35080358040271264

- FCTABLES format in HARP is conceived for NWP operational setups with runs i.e. at {00,06,12,18}UTC and lead_times of ~48-72h
- In OSVAS, FCTABLE production from long SURFEX OFFLINE runs emulates a SURFEX daily run at 00UTC with lead times up to +24h.

Step 4 (OPTIONAL): Sqlite extraction of 3d model data: grib2sqlite

- If grib files from a 3D-NWP run are available for the same validation period, it is possible to use [grib2sqlite](#) tool to extract the data to sqlites and add the model to the verification.
- This part is not yet included in the workflow. There is a bash script to do it:
[\\$OSVAS/scripts/bash_scripts/example_extract_sqlites_from_3dmodel.sh](#)

```
##### OSVAS Step 4 (Optional part #####
##### (OFFLINE SURFEX VALIDATION SYSTEM) #####
# EXTRACTION OF FC*SQLITE FILES interpolated from 3d MODEL #####
#####

OSVAS_HOME=$HOME/OSVAS                               #SET PATH TO YOUR OSVAS SETUP

#Give a name for your experiment, locate station & parameter list, output path,
#debug level, path & file naming scheme for input gribfiles, etc.
EXPERIMENT_NAME=AIBC_ibera
STATION_LIST=$OSVAS_HOME/sqlites/station_list_SURFEX.csv
PARAM_LIST=$OSVAS_HOME/config_files/HARP/param_list_SURFEX.csv
OUTPUT=$OSVAS_HOME/sqlites/data/
DEBUG_LEVEL=2
#Path to grib: wildcards allowed, current example is to extract only from 00 runs
PATH_TO_GRIBS=/ec/res4/scratch/esp0754/hm_home/AIBC_ibera_mc90_iasi_gnss_mdk/archive/*/*/*00/
GRIB_FILENAME_wildcards=fc*grib_sfxx
```

```
#Run the grib2sqlite tool with all provided arguments
```

```
python3 -m grib2sqlite \
-p $PARAM_LIST \
-s $STATION_LIST \
-o $OUTPUT \
-m $EXPERIMENT_NAME \
-d $DEBUG_LEVEL \
$PATH_TO_GRIBS/$GRIB_FILENAME_wildcards
```

- grib2sqlite uses another param_list to map grib variable names to HARP's variable names:

```
{ "harp_param": "H",
  "method": "bilin",
  "grib_id": { "shortName": "shf", "productDefinitionTemplateName": "0" }
},
{ "harp_param": "LE",
  "method": "bilin",
  "grib_id": { "shortName": "lhf", "productDefinitionTemplateName": "0" }
},
{ "harp_param": "Pmsl",
  "method": "bilin",
  "grib_id": { "shortName": "msl", "productDefinitionTemplateName": "0" }
},
```

Step 5: Configure a HARP verification (yaml file) and run HARP

- HARP verifications are configured also with a HARP yaml file.
- In this step, the template in **`$OSVAS/config_files/HARP/OSVAS_HARP_verif.yaml`** is used to generate a configuration file
`$OSVAS/config_files/HARP/OSVAS_HARP_verif_${STATION_NAME}.yaml`
- The template is filled with:
 - The list of Expnames,
 - Paths to the observations for the station,
 - Verification period read from `Validation_data.validation_start` and `Validation_data.validation_end`,
 - Output paths.
- The `point_verif.R` script is executed from the main script, with a list of variables extracted from the global yaml file (`Validation_data.variables`, left-hand side)
- Make sure that these variables are correctly described in
`${OSVAS}/config_files/HARP/set_params.R`

Step 5: Configure a HARP verification (yml file) and run HARP

- **set_params.R** can be used to homogeneize variable units in obs & fcst datasets, and to exclude from the analysis unrealistic variable ranges:

```
TS_4 = list(
  thresholds = c(-20, -10, seq(-5, 25, 5)),
  scale_fcst = list(scaling = -273.15, new_units = "degC"),
  scale_obs = list(scaling = 0, new_units = "degC"),
  obsmin_val = - 30,
  obsmax_val = + 50,
  error_sd = 6
),
TS_5 = list(
  thresholds = c(-20, -10, seq(-5, 25, 5)),
  scale_fcst = list(scaling = -273.15, new_units = "degC"),
  scale_obs = list(scaling = 0, new_units = "degC"),
  obsmin_val = - 30,
  obsmax_val = + 50,
  error_sd = 6
),
SWC_1 = list(
  thresholds = c(seq(0.025, 1.1, 0.025)),
  scale_fcst = list(scaling = 1, new_units = "m3/m3", mult = TRUE),
  scale_obs = list(scaling = 0.01, new_units = "m3/m3", mult = TRUE),
  obsmin_val = 0,
  obsmax_val = 10
),
SWC_2 = list(
  thresholds = c(seq(0.025, 1.1, 0.025)),
  scale_fcst = list(scaling = 1, new_units = "m3/m3", mult = TRUE),
  scale_obs = list(scaling = 0.01, new_units = "m3/m3", mult = TRUE),
  obsmin_val = 0,
  obsmax_val = 10
),
```

Step 6: Display HARP verification results

- Two shiny apps available:
 - app.R for “dynamic” inspection of HARP's output .rds data files
 - visapp.R for inspection of png figures with extra scores & visualizations.
- In a linux machine, both shiny apps will be available through a web browser in:
 - `http://127.0.0.1:9999` and
 - `http://127.0.0.1:9998` by default
- On ATOS machine (using ecmwf's virtual desktop through VMWare), I have only tested this step through the command line. An extra step is needed to access the shiny apps, with help shown in the script output:

```
[1] "To display the Shiny app in a Firefox window at ATOS:"  
[1] "1: Open a new terminal."  
[1] "2: Execute this command: ssh -L 9999:localhost:9999 "  
[1] "3: Open a Firefox window and go to http://127.0.0.1:9999/"
```


Namelists. For each site, we decided to use:

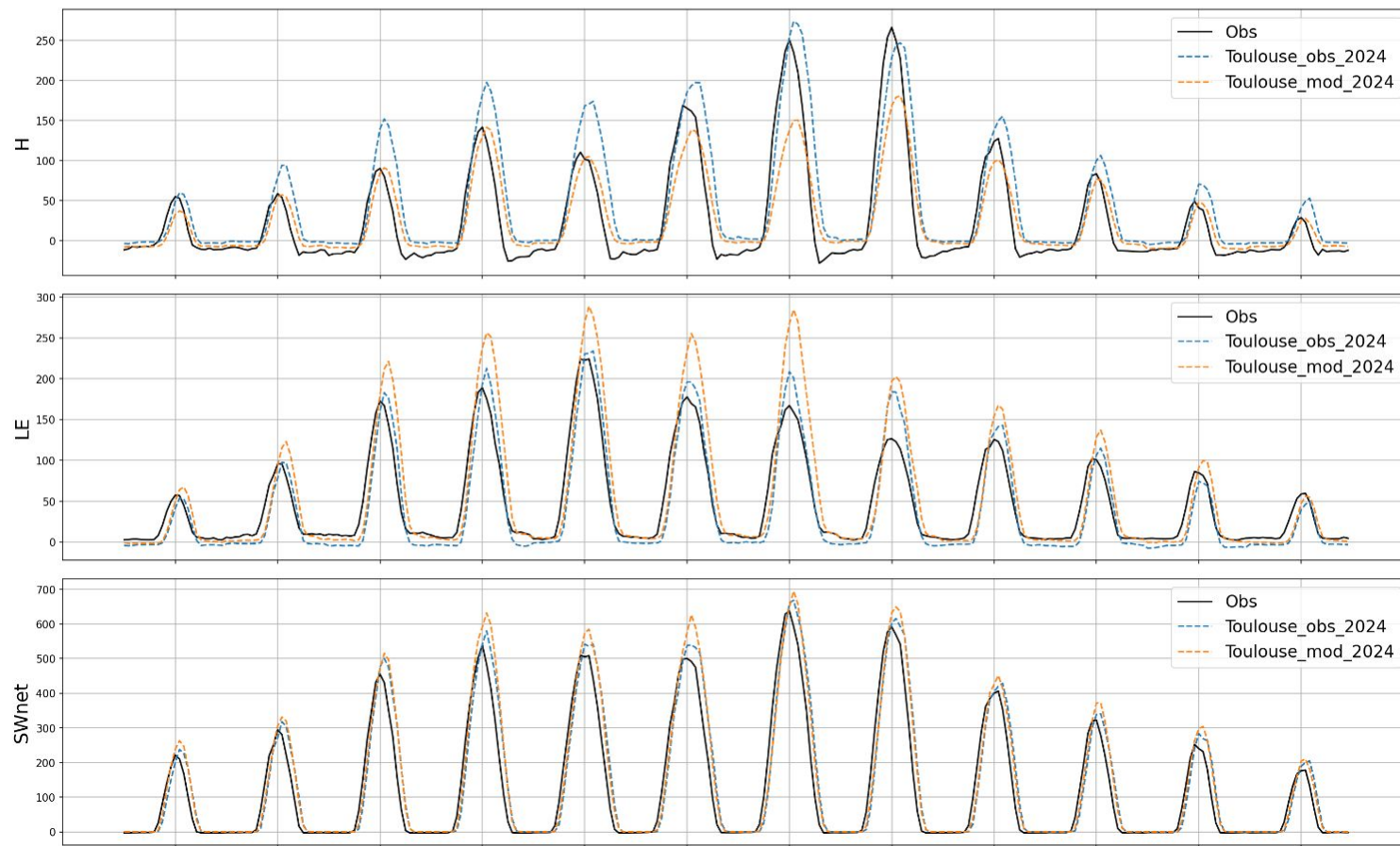
- **An “Ideal” namelist**, representing the local parameters as close as possible. It will typically have full number of patches (12 or 19), with the cover type fraction of the ecosystem 1, and all the rest 0.
- **A NWP-like namelist**, representing the target 3 patches setup for NWP, where the physiographic parameters come from the actual physiography used in our NWP setups: ECOCLIMAP-SG parameters, SOC, soil texture (SOILGRIDS), etc. This can be achieved by running PGD for every site on ATOS, BELENOS, etc (where all physiographic files are available) and extracting the site values to the namelist, or simply run all simulations under those HPCs.
- **(Optionally): An intermediate namelist**, with only one active patch like in the “ideal” namelist, but PGD extracted from the NWP physiography.

Not all three options are available for the current 3 stations, and they were collected during last winter, so there's also the need to update them with the current “common” configuration

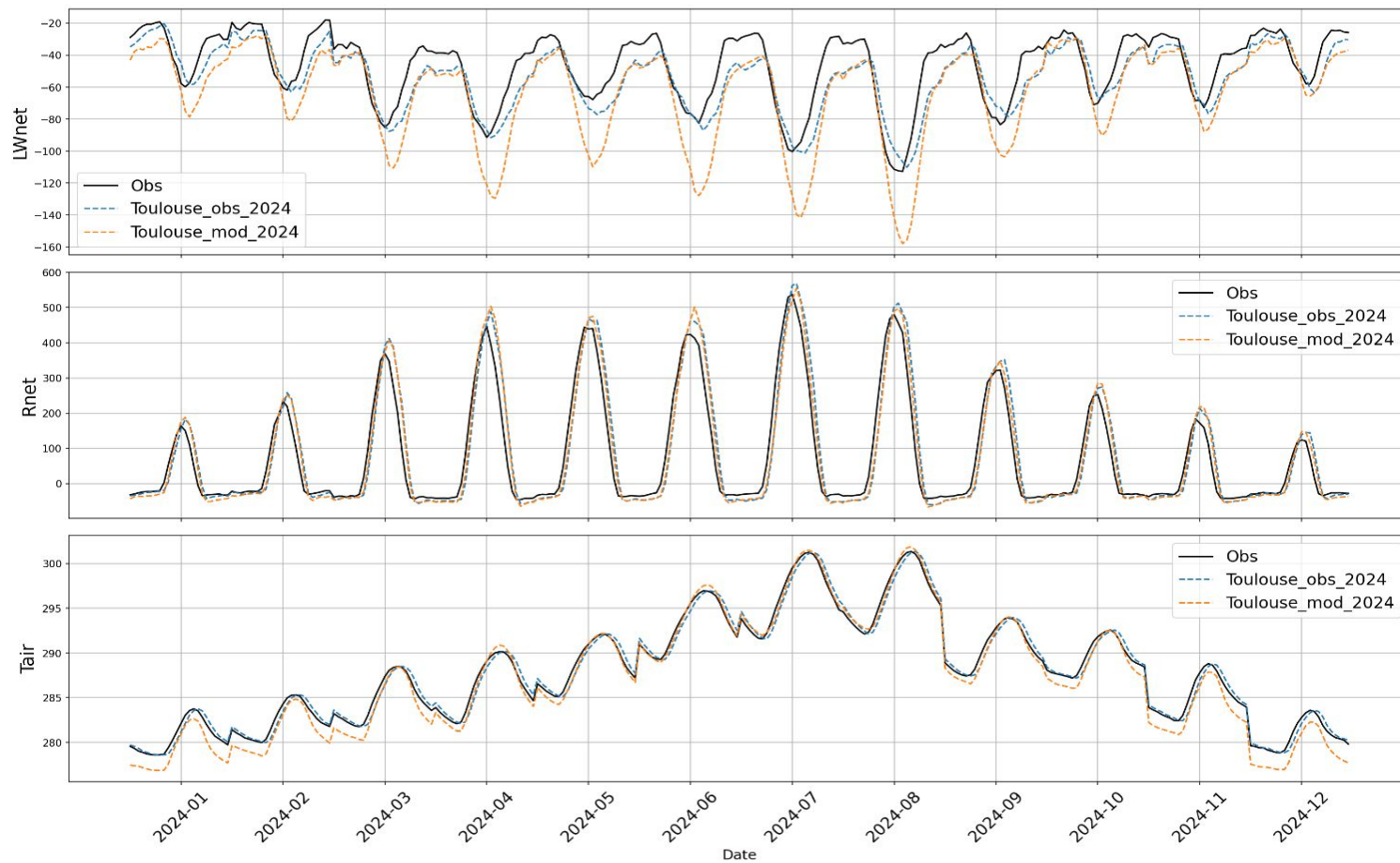
Development needs / ideas

- H & LE observations are not yet corrected to enforce SEB closure. A function is available in the notebook but not yet used, because currently there's no way to plot two sources of observations (original and SEB-corrected) in HARP plots
- Develop some automatic procedure to link DIF & soil layers in the yml files by relating the station's soil levels with the XSOILGRID vector.
- HARP scripts can't provide easily some plots (e.g. a monthly validation with results next to each other in the figures, see next 2 slides). This and other more sophisticated analysis could be incorporated to OSVAS by adapting already existing scripts in the SURFEX community.

Cycle diurne moyen mensuel — Obs vs Modèle (Toulouse)
de 2024-01-01 à 2024-12-30



Cycle diurne moyen mensuel — Obs vs Modèle (Toulouse)
de 2024-01-01 à 2024-12-30



Development needs / ideas

- Make it possible to run OSVAS workflow over any number of stations, all-at-once.
- Extend OSVAS to make it possible to validate a 2D offline SURFEX
- Extend 3d interpolation functionalities to work with FA files (e.g something like a FA2sqlite tool)
- **About model initialization (PREP):** Currently no initialization is done, i.e. a warm-up period is needed before the validation period. Ideas:
 - Initialize from ARPEGE? (Tested by Adrien)
 - Initialize from DEODE pan-european SURFEX OFFLINE setup
- Migrate the main control script to python, and/or incorporate pysurfex in the workflow to make it prepare the namelists, run the SURFEX steps & the rest of the current workflow?

Thanks for listening!